

# Pearls AQI Predictor

Karachi Air Quality Index Forecasting System



**Laiba Shabbir**

Bahria University, Karachi Campus

*Prepared: September 2026*

## PROJECT LINKS

**GitHub Repository:** <https://github.com/laibshab29/aqi-predictor>

**Live Dashboard:** <https://aqi-predictor-jisdw5uxskfcceuosxvvr3.streamlit.app/>

# 1. Project Overview

This project implements an end-to-end, automated system that forecasts the Air Quality Index (AQI) for Karachi, Pakistan, 24, 48, and 72 hours into the future. The system was built as a serverless-style pipeline: a feature pipeline collects live air quality and weather data every hour, a training pipeline retrains forecasting models daily on accumulated historical data, and an interactive Streamlit dashboard surfaces real-time readings, 3-day forecasts, and hazard alerts to the end user.

The four required final deliverables and where each is satisfied in this submission:

- **End-to-end AQI prediction system** — `feature_pipeline.py`, `training_pipeline.py`, and `app.py` together form a complete pipeline from raw data to a served prediction.
- **A scalable, automated pipeline** — two GitHub Actions workflows run the feature pipeline hourly and the training pipeline daily, requiring no manual intervention.
- **An interactive dashboard** — a public Streamlit app displays current AQI, a 3-day forecast chart, a hazard alert banner, and SHAP-based feature importance.
- **This report** — documenting the architecture, data source, methodology, results, and honest limitations of the system.

## 2. System Architecture

The system follows a four-stage pipeline, illustrated below:

Open-Meteo API → Feature Pipeline → Feature Store (`data/features.csv`) → Training Pipeline →  
Model Registry (`models/*.joblib`) → Streamlit Dashboard

Automation is layered on top via two scheduled GitHub Actions workflows: one runs the feature pipeline every hour to keep the feature store current, and the other retrains the models daily as more historical data accumulates.

## 3. Data Source

All data in this project — both the historical training set and the live hourly readings — comes from the Open-Meteo Air Quality and Historical Weather APIs. This source was chosen deliberately over alternatives such as AQICN because it is free, requires no API key or account registration, and provides genuine historical hourly data back to August 2022, unlike free-tier alternatives which typically expose only the current reading.

Using the same data source for both training and live inference also avoids "train/serve skew" — a common real-world ML pipeline bug where the model is trained on data computed one way, but served predictions on data computed a slightly different way, silently degrading accuracy.

The features used are: hour of day, day of month, month, day of week (time-based features), PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, and ozone concentrations, temperature,

humidity, atmospheric pressure, wind speed, and the AQI change rate (a derived feature capturing the difference between the current and previous AQI reading).

## 4. Modeling Approach

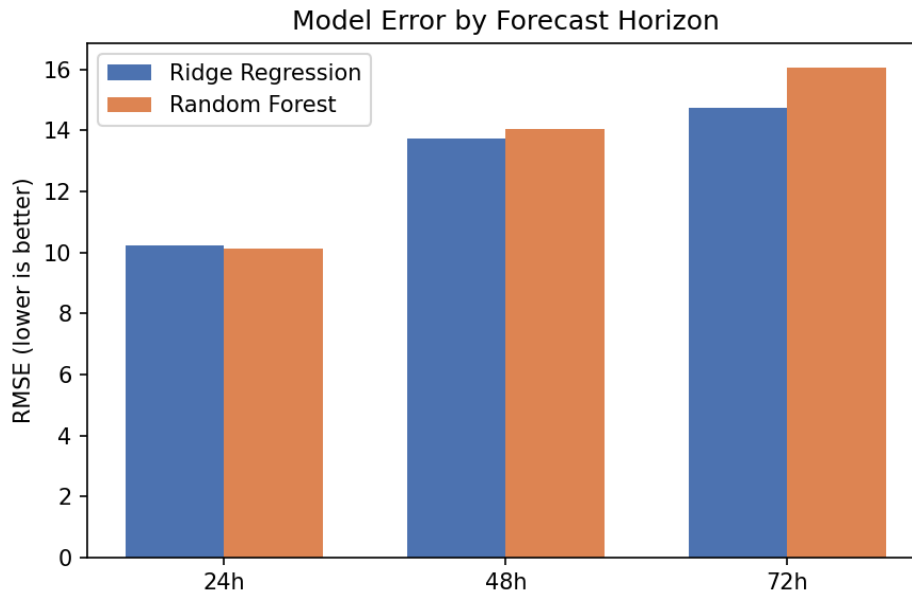
Three separate forecasting targets were created — AQI 24, 48, and 72 hours ahead — since a single-step model cannot natively produce a multi-day forecast. For each horizon, two models were trained and compared: Ridge Regression, as a simple linear baseline, and Random Forest Regression, to capture non-linear relationships such as rush-hour pollution spikes and seasonal patterns. Data was split chronologically (80% earliest data for training, 20% most recent data for testing) rather than randomly, since air quality is a time series and a random split would leak future information into training.

The better-performing model per horizon (by RMSE) was automatically selected and saved to the model registry. SHAP (SHapley Additive exPlanations) was used to generate feature importance plots for the Random Forest models, allowing the dashboard to explain which inputs most influenced each forecast.

## 5. Results

The models were trained on 23,496 real hourly records collected via the Open-Meteo backfill script, covering roughly January 2024 through September 2026. Evaluation metrics on the held-out most-recent 20% of data:

| Horizon | Model         | RMSE  | MAE   | R-sq   | Selected |
|---------|---------------|-------|-------|--------|----------|
| 24h     | Ridge         | 10.23 | 7.31  | 0.507  |          |
| 24h     | Random Forest | 10.11 | 6.89  | 0.519  | ✓        |
| 48h     | Ridge         | 13.72 | 10.16 | 0.111  | ✓        |
| 48h     | Random Forest | 14.06 | 9.97  | 0.067  |          |
| 72h     | Ridge         | 14.73 | 10.89 | -0.028 | ✓        |
| 72h     | Random Forest | 16.05 | 11.08 | -0.220 |          |



The 24-hour model performs reasonably well, explaining roughly 52% of the variance in next-day AQI ( $R^2 = 0.519$ ). Accuracy drops substantially at 48 and 72 hours, with the 72-hour model's  $R^2$  turning slightly negative — meaning it performs marginally worse than simply predicting the historical average. This is discussed honestly in the Limitations section below rather than concealed, as it reflects a genuine and explainable constraint of the current approach.

## 6. Dashboard

The Streamlit dashboard, deployed publicly and linked on the title page, displays: the current AQI reading with its severity category; a bar chart comparing current AQI against the 24h/48h/72h forecasts; a hazard alert banner that turns red whenever any forecasted value crosses the "Unhealthy" threshold ( $AQI \geq 151$ ); a detailed forecast table with category labels; and SHAP feature-importance charts per horizon, so a user can see which factors (e.g. PM2.5, hour of day) drove a given prediction.

Because the underlying feature store and model registry are updated automatically by the scheduled GitHub Actions workflows, the deployed dashboard reflects fresh data without any manual redeployment.

## 7. Limitations and Future Work

- **Forecast accuracy degrades sharply beyond 24 hours.** The current models only see the present hour's weather and pollution readings, not a genuine weather forecast for the target time. A meaningful improvement would be to incorporate Open-Meteo's own weather *\*forecast\** endpoint (as opposed to current conditions) as an input feature for the 48h/72h models, since future temperature/humidity/wind are themselves forecastable and informative for future AQI.
- **No lag/rolling features yet.** Adding features such as the AQI trend over the past 6-24 hours (not just the single most recent change) would likely improve medium-range accuracy, since pollution episodes tend to build up or dissipate over multiple hours.

- **No deep learning model has been tried yet.** An LSTM or similar sequence model over the hourly time series is a natural next step, and may better capture the temporal dependencies that a per-row Random Forest cannot.
- **Single-city scope.** The system is currently hard-coded to Karachi's coordinates; generalizing to multiple cities would require parameterizing the pipeline and retraining per-city models.
- **Feature store and model registry are file-based (CSV/joblib), not a managed service like Hopsworks.** This was a deliberate scope decision to prioritize a working, fully automated end-to-end system within the project timeline. The codebase isolates these concerns behind small functions (save\_feature\_row, model loading/saving) specifically so they can be swapped for a managed feature store and model registry with minimal changes elsewhere.

## 8. Conclusion

This project delivers a fully automated, end-to-end AQI forecasting system for Karachi, built entirely on free and open data sources, with no manual steps required after initial setup. Short-term (24h) forecasts are reasonably reliable, while medium-term forecasts highlight a clear and well-understood direction for future improvement. The architecture was intentionally kept modular so that individual components — the data source, the feature store, the model type, and the forecast horizon — can each be upgraded independently as the project continues.